

End-to-End RAG on Real Documents

Project Summary & Interview Q&A

A complete Retrieval-Augmented Generation pipeline: PDF ingestion → chunking → embedding → FAISS indexing → semantic retrieval → LLM generation → faithfulness evaluation.

Tech stack: Python · PyPDF2 · sentence-transformers · FAISS · OpenAI API · NumPy

Pipeline Architecture

The system processes a PDF through six sequential stages. Each stage is isolated in its own module so components can be swapped independently.

Stage	Module	Key Function	Library
1 · Ingestion	ingestion/pdf_loader.py	load_pdf(file_path)	PyPDF2
	ingestion/chunking.py	chunk_text(text, size=500, overlap=100)	—
2 · Embedding	embeddings/embedder.py	get_embeddings(chunks)	sentence-transformers
3 · Indexing	retrieval/faiss_index.py	create_faiss_index(embeddings)	FAISS
4 · Retrieval	retrieval/retrieval.py	retrieve(query, model, index, chunks, k=3)	FAISS
5 · Generation	generation/llm_pipeline.py	generate_answer(query, context_chunks)	OpenAI API
6 · Evaluation	evaluation/metrics.py	evaluate(answer, context, embed_fn)	NumPy

Data Flow

- load_pdf** extracts raw text from every page via PyPDF2.
- chunk_text** splits the text into 500-char windows with 100-char overlap.
- get_embeddings** encodes each chunk into a 384-d float32 vector.
- create_faiss_index** adds all vectors into an IndexFlatL2 index.
- At query time, **retrieve** encodes the question, runs a k-NN search (k=3), and returns the nearest chunks as context.
- generate_answer** sends the context + question to GPT-3.5-Turbo (temp=0).
- evaluate** scores the answer for faithfulness via cosine similarity.

Running the App

```
pip install -r requirements.txt
export OPENAI_API_KEY=sk-...
python app.py path/to/document.pdf
```

Interview Questions & Answers

Q: [1] What is Retrieval-Augmented Generation (RAG)?

A: RAG is an AI framework that combines a retrieval system with a generative LLM. Instead of relying solely on the model's parametric knowledge, RAG first retrieves relevant passages from an external document store and injects them into the prompt, allowing the model to ground its answer in up-to-date, domain-specific content.

Q: [2] Why use RAG instead of fine-tuning?

A: Fine-tuning embeds knowledge into model weights — expensive, slow to update, and prone to catastrophic forgetting. RAG externalises knowledge into a retrieval index that can be updated cheaply without retraining the model. RAG also provides citations and is more transparent about its information source.

Q: [3] What problem does chunking solve, and what are the tradeoffs?

A: LLMs have a fixed context window, so we cannot feed an entire document at once. Chunking splits the document into pieces small enough to embed and compare. Smaller chunks = more precise retrieval but lose surrounding context. Larger chunks = richer context per hit but noisier similarity scores. The overlap parameter (100 chars here) prevents information loss at chunk boundaries.

Q: [4] Which embedding model is used and why?

A: 'all-MiniLM-L6-v2' from sentence-transformers. It is a lightweight (22 M params) transformer that produces 384-dimensional unit-normalised embeddings. It balances speed and quality well for English semantic search tasks and is freely available offline — no API call needed for embedding.

Q: [5] Why is FAISS IndexFlatL2 equivalent to cosine similarity here?

A: Cosine similarity = dot product of unit vectors. The MiniLM model returns L2-normalised embeddings ($\|v\| = 1$). For unit vectors, $\|a - b\|^2 = 2 - 2 \cdot (a \cdot b)$, which is a monotonic transformation of cosine similarity. Ranking by L2 distance therefore gives the same ordering as ranking by cosine similarity — no separate normalisation step needed.

Q: [6] What does IndexFlatL2 do internally?

A: It performs an exact brute-force L2 distance scan over all stored vectors. It makes no approximation, so recall = 100%, but query time scales linearly $O(n \cdot d)$ with the number of vectors n and dimension d . For large corpora, approximate indexes like IVFFlat or HNSW are preferred.

Q: [7] How does retrieval work at query time?

A: 1. The user query is encoded with the same SentenceTransformer model into a 384-d vector. 2. FAISS performs an L2 nearest-neighbour search and returns the $k=3$ closest chunk vectors. 3. The corresponding text chunks are assembled as context and passed to the LLM alongside the original question.

Q: [8] What prompt strategy is used for generation?

A: A two-message chat prompt: a system message instructing the model to answer only from the provided context and to admit uncertainty when context is insufficient, and a user message containing the retrieved context blocks followed by the question. Temperature is set to 0 for deterministic, factual answers.

Q: [9] Explain the faithfulness metric.

A: Faithfulness measures how semantically consistent the generated answer is with the retrieved context. Both the answer and the concatenated context are embedded with the same SentenceTransformer model, then their cosine similarity is computed. A score near 1 means the answer closely reflects the context; a low score may indicate hallucination.

Q: [10] What is retrieval_precision in the evaluation module?

A: It is the fraction of retrieved chunks that contain (or are contained in) at least one known-relevant text snippet. It requires a ground-truth list of relevant passages. $\text{Precision} = |\text{relevant} \cap \text{retrieved}| / |\text{retrieved}|$. This is useful for offline evaluation when labelled data is available.

Q: [11] What are the limitations of this RAG implementation?

A: 1. Character-level chunking can split words and sentences. 2. IndexFlatL2 does not scale past ~100 k vectors without slowdown. 3. No persistent index — it is rebuilt on every run. 4. Single-document only; multi-document retrieval is not handled. 5. Evaluation metrics are basic; no BLEU/ROUGE or human eval. 6. No re-ranking step after initial retrieval.

Q: [12] How would you scale this to production?

A: Replace IndexFlatL2 with an IVFFlat or HNSW index and persist it to disk. Add a metadata store (SQLite/PostgreSQL) to map chunk IDs back to source documents and page numbers. Introduce a re-ranker (cross-encoder) between retrieval and generation to improve precision. Use a vector database (Pinecone, Weaviate, pgvector) for managed, scalable storage. Cache embeddings so re-indexing only processes changed documents.

Q: [13] How would you improve chunking quality?

A: Switch from character-level windows to sentence-aware splitting (e.g., using spaCy or NLTK sentence tokenisation), or semantic chunking that groups sentences by topic similarity. Hierarchical chunking (large parent + small child chunks) can also be used: retrieve small chunks for precision, but send the parent chunk to the LLM for context richness.

Q: [14] What is the role of the overlap parameter?

A: Overlap ensures that information near chunk boundaries is not lost. With `chunk_size=500` and `overlap=100`, consecutive chunks share 100 characters. This means a sentence split across two chunks will appear in full in at least one of them, improving recall for boundary-spanning queries.

Q: [15] How is the SentenceTransformer model loaded, and why does it matter?

A: It is loaded as a module-level global in `embedder.py` — once on first import. This is efficient for a single-process pipeline because the model (~80 MB) is loaded exactly once. In a multi-worker web server, each worker process would load its own copy; for that case, a shared model server (Triton, TorchServe) or lazy singleton with a lock would be better.

Q: [16] What would you add to make this system production-ready?

A: Async query handling, streaming LLM responses, document-level access control, query logging and feedback collection for continuous improvement, a REST API layer (FastAPI), a simple frontend (Streamlit or React), automated re-indexing on document upload, and integration tests with a fixed document and known ground-truth answers to catch regressions.